

Lab Exercise

Big Data Processing with PySpark (Retail Transaction Analytics)

Case Study: Spark for Retail Analytics

1. Background

You are a **Data Analyst** in an international e-commerce company handling large-scale retail transactions across multiple countries. The company collects data such as customer purchases, product details, transaction time, and geographic information to support business decisions.

2. Problem Statement

Previously, the company used **Hadoop MapReduce** for data processing. However, several limitations have been identified:

- **Slow performance** due to disk-based processing
- Difficulty in performing **iterative and interactive analysis**
- Delayed business insights and decision-making

To address these issues, the company is transitioning to **Apache Spark**, which enables faster, in-memory data processing and supports advanced analytics.

3. Data Challenge

The dataset provided is **real-world and contains dirty data**, including:

- Missing values (e.g., CustomerID)
- Invalid values (e.g., negative quantity, zero price)
- Duplicate records
- Inconsistent formats (e.g., product descriptions, timestamps)

You must identify and handle these issues before performing analysis.

Note:

This lab exercise uses a real dataset provided by the University of California, Irvine (UCI) Machine Learning Repository: <https://archive.ics.uci.edu/>

PART A — Environment Setup

Step 1. Open Google Colab (faster than using Jupyter on our Docker)

Step 2. Install PySpark

```
!pip install pyspark
```

Step 3. Initialize Spark

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("RetailBigDataPipeline") \
    .getOrCreate()
```

Step 4. Upload Dataset

```
from google.colab import files
uploaded = files.upload()
```

Step 5. Load Data

```
df = spark.read.csv("OnlineRetail.csv", header=True, inferSchema=True)
df.show(5)
```

PART B — DATA UNDERSTANDING & DIRTY DATA

Step 6. Explore Data

```
df.printSchema()
df.describe().show()
```

Step 7. Identify Dirty Data

```
from pyspark.sql.functions import col, count, when

df.select([
    count(when(col(c).isNull(), c)).alias(c)
    for c in df.columns
]).show()
```

Step 8. Feature Engineering and basic cleaning

```
from pyspark.sql.functions import col

df = df.dropna(subset=["CustomerID"])
df = df.filter(col("Quantity") != 0)
df = df.filter(col("UnitPrice") > 0)
df = df.withColumn("TotalAmount", col("Quantity") * col("UnitPrice"))
```

PART C — SPARK PROCESSING

C1 — Sales by Country

```
from pyspark.sql.functions import sum as _sum, round as _round

sales_by_country = (
    df.groupBy("Country")
        .agg(_round(_sum("TotalAmount"), 2).alias("TotalSales"))
        .orderBy(col("TotalSales").desc())
)

sales_by_country.show(20, truncate=False)
```

C2 — Top Products

```
from pyspark.sql.functions import sum as _sum, count as _count, round
as _round

top_products = (
    df.groupBy("StockCode", "Description")
        .agg(
            _sum("Quantity").alias("TotalUnitsSold"),
            _round(_sum("TotalAmount"), 2).alias("TotalRevenue"),
            _count("*").alias("TransactionCount")
        )
        .orderBy(col("TotalUnitsSold").desc())
)

top_products.show(10, truncate=False)
```

Alternative: top products by revenue

```
top_products_by_revenue = (
    df.groupBy("StockCode", "Description")
        .agg(_round(_sum("TotalAmount"), 2).alias("TotalRevenue"))
        .orderBy(col("TotalRevenue").desc())
)

top_products_by_revenue.show(10, truncate=False)
```

What it does:

Finds the most important products by quantity sold and optionally by revenue.

C3 — Customer Behavior**A. Most active customers**

```
from pyspark.sql.functions import countDistinct

most_active_customers = (
    df.groupBy("CustomerID")
      .agg(
        count("*").alias("TransactionLines"),
        countDistinct("InvoiceNo").alias("UniqueInvoices")
      )
      .orderBy(col("UniqueInvoices").desc(),
        col("TransactionLines").desc())
)

most_active_customers.show(10, truncate=False)
```

B. Highest spending customers

```
highest_spending_customers = (
    df.groupBy("CustomerID")
      .agg(
        round(sum("TotalAmount"), 2).alias("TotalSpent"),
        countDistinct("InvoiceNo").alias("UniqueInvoices")
      )
      .orderBy(col("TotalSpent").desc())
)

highest_spending_customers.show(10, truncate=False)
```

C. Average spending per customer

```
from pyspark.sql.functions import avg

avg_spending_per_customer = (
    df.groupBy("CustomerID")
      .agg(
        round(avg("TotalAmount"), 2).alias("AverageLineSpending")
      )
      .orderBy(col("AverageLineSpending").desc())
)

avg_spending_per_customer.show(10, truncate=False)
```

What it does:

Shows customer activity and spending patterns, useful for identifying loyal or high-value customers.

C4 — Time-Based Analysis

First, convert the invoice date into proper date/time fields.

```
from pyspark.sql.functions import to_timestamp, to_date, year, month,
dayofmonth, hour

df_time = df.withColumn(
    "InvoiceTimestamp",
    to_timestamp(col("InvoiceDate"), "M/d/yyyy H:mm")
)

df_time = df_time.withColumn("InvoiceDay",
    to_date(col("InvoiceTimestamp")))
df_time = df_time.withColumn("Year", year(col("InvoiceTimestamp")))
df_time = df_time.withColumn("Month", month(col("InvoiceTimestamp")))
df_time = df_time.withColumn("Day",
    dayofmonth(col("InvoiceTimestamp")))
df_time = df_time.withColumn("Hour", hour(col("InvoiceTimestamp")))
```

A. Daily sales

```
from pyspark.sql.functions import to_timestamp, to_date, year, month,
dayofmonth, hour, col, sum as _sum, count as _count, round as _round

# Re-create df_time with corrected timestamp parsing
df_time = df.withColumn(
    "InvoiceTimestamp",
    to_timestamp(col("InvoiceDate"), "d/M/yyyy H:mm")
)

df_time = df_time.withColumn("InvoiceDay",
    to_date(col("InvoiceTimestamp")))
df_time = df_time.withColumn("Year", year(col("InvoiceTimestamp")))
df_time = df_time.withColumn("Month", month(col("InvoiceTimestamp")))
df_time = df_time.withColumn("Day",
    dayofmonth(col("InvoiceTimestamp")))
df_time = df_time.withColumn("Hour", hour(col("InvoiceTimestamp")))

daily_sales = (
    df_time.groupBy("InvoiceDay")
        .agg(
            _round(_sum("TotalAmount"), 2).alias("DailySales"),
            _count("*").alias("TransactionLines")
        )
        .orderBy("InvoiceDay")
)

daily_sales.show(20, truncate=False)
```

B. Monthly sales

```
monthly_sales = (
    df_time.groupBy("Year", "Month")
```

```

        .agg(
            _round(_sum("TotalAmount"), 2).alias("MonthlySales"),
            _count("*").alias("TransactionLines")
        )
        .orderBy("Year", "Month")
    )

monthly_sales.show(20, truncate=False)

```

C. Hourly transaction pattern

```

hourly_pattern = (
    df_time.groupBy("Hour")
        .agg(
            _count("*").alias("TransactionLines"),
            _round(_sum("TotalAmount"), 2).alias("HourlySales")
        )
        .orderBy("Hour")
    )

hourly_pattern.show(24, truncate=False)

```

What it does:

Adds a temporal dimension so students can analyze patterns by day, month, and hour.

C5 — Fraud Simulation / Suspicious Transaction Logic

Because the original Online Retail dataset does not contain a fraud label, we create a **rule-based suspicious flag**.

Below is one example rule:

A transaction line is suspicious if:

- **Quantity** >= 500, **or**
- **TotalAmount** >= 10000

```

from pyspark.sql.functions import when

df_suspicious = df_time.withColumn(
    "SuspiciousFlag",
    when((col("Quantity") >= 500) | (col("TotalAmount") >= 10000),
1).otherwise(0)
)

```

A. Count suspicious vs normal

```
suspicious_summary = (
    df_suspicious.groupBy("SuspiciousFlag")
                  .count()
                  .orderBy("SuspiciousFlag")
)

suspicious_summary.show()
```

B. Show suspicious transactions

```
suspicious_transactions = (
    df_suspicious.filter(col("SuspiciousFlag") == 1)
                  .select(
                      "InvoiceNo", "CustomerID", "StockCode",
                      "Description",
                      "Quantity", "UnitPrice", "TotalAmount", "Country",
                      "InvoiceTimestamp", "SuspiciousFlag"
                  )
                  .orderBy(col("TotalAmount").desc())
)

suspicious_transactions.show(20, truncate=False)
```

C. Suspicious transactions by country

```
suspicious_by_country = (
    df_suspicious.groupBy("Country")
                  .agg(
                      _count(when(col("SuspiciousFlag") == 1,
                      True)).alias("SuspiciousCount"),
                      _count("*").alias("TotalTransactions")
                  )
                  .orderBy(col("SuspiciousCount").desc())
)

suspicious_by_country.show(20, truncate=False)
```

D. Suspicious customers

```
suspicious_customers = (
    df_suspicious.filter(col("SuspiciousFlag") == 1)
                  .groupBy("CustomerID")
                  .agg(
                      _count("*").alias("SuspiciousTransactions"),
                      _round(_sum("TotalAmount"),
                      2).alias("SuspiciousValue")
                  )
                  .orderBy(col("SuspiciousTransactions").desc(),
                      col("SuspiciousValue").desc())
)

suspicious_customers.show(20, truncate=False)
```

What it does:

Creates a simple anomaly-detection style task so students can justify their own suspicious-transaction rules.

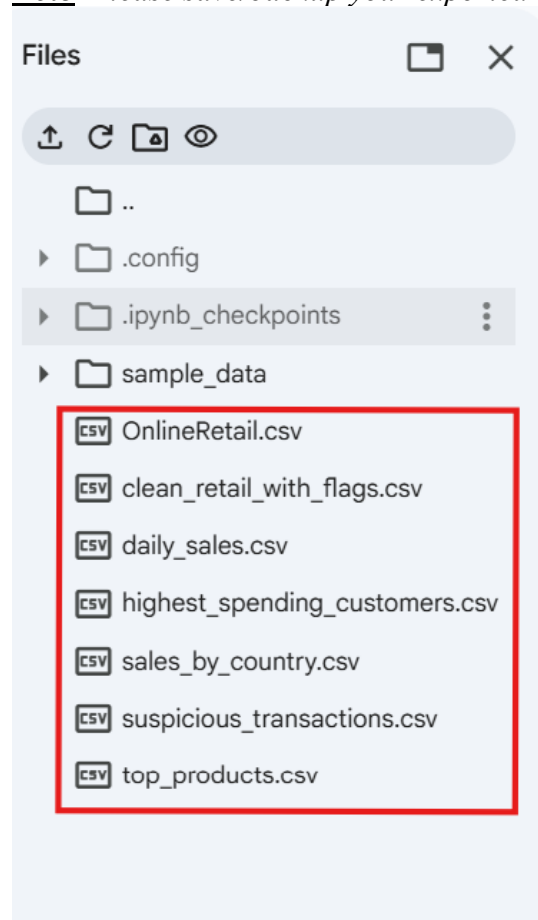
Export cleaned transactional data

```
df_suspicious.toPandas().to_csv("clean_retail_with_flags.csv",  
index=False)
```

Export summary tables

```
sales_by_country.toPandas().to_csv("sales_by_country.csv", index=False)  
top_products.toPandas().to_csv("top_products.csv", index=False)  
highest_spending_customers.toPandas().to_csv("highest_spending_customer  
s.csv", index=False)  
daily_sales.toPandas().to_csv("daily_sales.csv", index=False)  
suspicious_transactions.toPandas().to_csv("suspicious_transactions.csv"  
, index=False)
```

Note: Please save/backup your exported dataset. We will use this dataset for next week's lab.



Next week Lab: Dashboard Requirements

1. KPI:
 - Total Sales
 - Total Transactions
2. Visualizations:
 - Sales by Country (Bar Chart)
 - Top Products (Bar Chart)
 - Sales Trend (Line Chart)
 - Customer Segmentation
3. Insight Panel:
 - 3 business insights

To answer:

- Which country generates highest revenue?
- Which product dominates sales?
- What pattern exists over time?
- What strategy should the company implement?

And to address:

1. Difference MapReduce vs Spark

Explain based on lab experience

2. Is Spark faster than MapReduce? Why?

3. Transformation vs Action

4. Identify Business Insight

5. Perform Real Analytics

6. Big Data Pipeline Evolution

To explain:

HDFS → MapReduce → Spark → BI Dashboard